

Đây là notebook đi kèm cho cuốn sách [Deep Learning with Python, Third Edition](#). Để đảm bảo tính dễ đọc, notebook này chỉ chứa các khối mã có thể chạy được và tiêu đề các phần, đồng thời lược bỏ các nội dung khác trong sách như: các đoạn văn bản, hình ảnh và mã giả.

Nếu bạn muốn theo dõi những gì đang diễn ra, tôi khuyên bạn nên đọc notebook này song song với bản sao của cuốn sách.

Nội dung cuốn sách có sẵn trực tuyến tại deeplearningwithpython.io.

```
!pip install keras keras-hub --upgrade -q
```

```
import os
os.environ["KERAS_BACKEND"] = "jax"
```

[Hiện mã](#)

✖ Phân loại và Hồi quy

✖ Phân loại các đánh giá phim: Một ví dụ về phân loại nhị phân (binary classification)

✖ Tập dữ liệu IMDb

```
from keras.datasets import imdb

(train_data, train_labels), (test_data, test_labels) = imdb.load_data(
    num_words=10000
)
```

```
train_data[0]
```

```
train_labels[0]
```

```
max([max(sequence) for sequence in train_data])
```

```
word_index = imdb.get_word_index()
reverse_word_index = dict([(value, key) for (key, value) in word_index.items()])
decoded_review = " ".join(
    [reverse_word_index.get(i - 3, "?") for i in train_data[0]]
)
```

```
decoded_review[:100]
```

✖ Chuẩn bị dữ liệu

```
import numpy as np

def multi_hot_encode(sequences, num_classes):
    results = np.zeros((len(sequences), num_classes))
    for i, sequence in enumerate(sequences):
        results[i][sequence] = 1.0
    return results

x_train = multi_hot_encode(train_data, num_classes=10000)
x_test = multi_hot_encode(test_data, num_classes=10000)
```

```
x_train[0]
```

```
y_train = train_labels.astype("float32")
y_test = test_labels.astype("float32")
```

✖ Xây dựng mô hình

```
import keras
from keras import layers

model = keras.Sequential(
    [
        layers.Dense(16, activation="relu"),
        layers.Dense(16, activation="relu"),
        layers.Dense(1, activation="sigmoid"),
    ]
)
```

```
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"],
)
```

▼ Kiểm chứng phương pháp của bạn (Validating your approach)

```
x_val = x_train[:10000]
partial_x_train = x_train[10000:]
y_val = y_train[:10000]
partial_y_train = y_train[10000:]
```

```
history = model.fit(
    partial_x_train,
    partial_y_train,
    epochs=20,
    batch_size=512,
    validation_data=(x_val, y_val),
)
```

```
history = model.fit(
    x_train,
    y_train,
    epochs=20,
    batch_size=512,
    validation_split=0.2,
)
```

```
history_dict = history.history
history_dict.keys()
```

```
import matplotlib.pyplot as plt

history_dict = history.history
loss_values = history_dict["loss"]
val_loss_values = history_dict["val_loss"]
epochs = range(1, len(loss_values) + 1)
plt.plot(epochs, loss_values, "r--", label="Training loss")
plt.plot(epochs, val_loss_values, "b", label="Validation loss")
plt.title("[IMDB] Training and validation loss")
plt.xlabel("Epochs")
plt.xticks(epochs)
plt.ylabel("Loss")
plt.legend()
plt.show()
```

```
plt.clf()
acc = history_dict["accuracy"]
val_acc = history_dict["val_accuracy"]
plt.plot(epochs, acc, "r--", label="Training acc")
plt.plot(epochs, val_acc, "b", label="Validation acc")
plt.title("[IMDB] Training and validation accuracy")
plt.xlabel("Epochs")
plt.xticks(epochs)
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

```

model = keras.Sequential(
    [
        layers.Dense(16, activation="relu"),
        layers.Dense(16, activation="relu"),
        layers.Dense(1, activation="sigmoid"),
    ]
)
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"],
)
model.fit(x_train, y_train, epochs=4, batch_size=512)
results = model.evaluate(x_test, y_test)

```

results

- ▼ Sử dụng mô hình đã huấn luyện để tạo dự đoán trên dữ liệu mới

```
model.predict(x_test)
```

Các thử nghiệm mở rộng

Tổng kết

- ▼ Phân loại tin vắn: Một ví dụ về phân loại đa lớp (multiclass classification)
- ▼ Tập dữ liệu Reuters

```

from keras.datasets import reuters

(train_data, train_labels), (test_data, test_labels) = reuters.load_data(
    num_words=10000
)

```

```
len(train_data)
```

```
len(test_data)
```

```
train_data[10]
```

```

word_index = reuters.get_word_index()
reverse_word_index = dict([(value, key) for (key, value) in word_index.items()])
decoded_newswire = " ".join(
    [reverse_word_index.get(i - 3, "?") for i in train_data[10]]
)

```

```
train_labels[10]
```

- ▼ Chuẩn bị dữ liệu

```

x_train = multi_hot_encode(train_data, num_classes=10000)
x_test = multi_hot_encode(test_data, num_classes=10000)

```

```

def one_hot_encode(labels, num_classes=46):
    results = np.zeros((len(labels), num_classes))
    for i, label in enumerate(labels):
        results[i, label] = 1.0
    return results

y_train = one_hot_encode(train_labels)
y_test = one_hot_encode(test_labels)

```

```
from keras.utils import to_categorical

y_train = to_categorical(train_labels)
y_test = to_categorical(test_labels)
```

✦ Xây dựng mô hình

```
model = keras.Sequential(
    [
        layers.Dense(64, activation="relu"),
        layers.Dense(64, activation="relu"),
        layers.Dense(46, activation="softmax"),
    ]
)
```

```
top_3_accuracy = keras.metrics.TopKCategoricalAccuracy(
    k=3, name="top_3_accuracy"
)
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy", top_3_accuracy],
)
```

✦ Kiểm chứng phương pháp của bạn

```
x_val = x_train[:1000]
partial_x_train = x_train[1000:]
y_val = y_train[:1000]
partial_y_train = y_train[1000:]
```

```
history = model.fit(
    partial_x_train,
    partial_y_train,
    epochs=20,
    batch_size=512,
    validation_data=(x_val, y_val),
)
```

```
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(loss) + 1)
plt.plot(epochs, loss, "r--", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.xlabel("Epochs")
plt.xticks(epochs)
plt.ylabel("Loss")
plt.legend()
plt.show()
```

```
plt.clf()
acc = history.history["accuracy"]
val_acc = history.history["val_accuracy"]
plt.plot(epochs, acc, "r--", label="Training accuracy")
plt.plot(epochs, val_acc, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.xlabel("Epochs")
plt.xticks(epochs)
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

```
plt.clf()
acc = history.history["top_3_accuracy"]
val_acc = history.history["val_top_3_accuracy"]
plt.plot(epochs, acc, "r--", label="Training top-3 accuracy")
plt.plot(epochs, val_acc, "b", label="Validation top-3 accuracy")
plt.title("Training and validation top-3 accuracy")
plt.xlabel("Epochs")
```

```
plt.xticks(epochs)
plt.ylabel("Top-3 accuracy")
plt.legend()
plt.show()
```

```
model = keras.Sequential(
    [
        layers.Dense(64, activation="relu"),
        layers.Dense(64, activation="relu"),
        layers.Dense(46, activation="softmax"),
    ]
)
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
model.fit(
    x_train,
    y_train,
    epochs=9,
    batch_size=512,
)
results = model.evaluate(x_test, y_test)
```

results

```
import copy
test_labels_copy = copy.copy(test_labels)
np.random.shuffle(test_labels_copy)
hits_array = np.array(test_labels == test_labels_copy)
hits_array.mean()
```

▼ Tạo dự đoán trên dữ liệu mới

```
predictions = model.predict(x_test)
```

```
predictions[0].shape
```

```
np.sum(predictions[0])
```

```
np.argmax(predictions[0])
```

▼ Một cách khác để xử lý nhãn và hàm mất mát (loss function)

```
y_train = train_labels
y_test = test_labels
```

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

▼ Tầm quan trọng của việc có các lớp trung gian đủ lớn

```
model = keras.Sequential(
    [
        layers.Dense(64, activation="relu"),
        layers.Dense(4, activation="relu"),
        layers.Dense(46, activation="softmax"),
    ]
)
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
```

```
)
model.fit(
    partial_x_train,
    partial_y_train,
    epochs=20,
    batch_size=128,
    validation_data=(x_val, y_val),
)
```

Các thử nghiệm mở rộng

Tổng kết

- ✓ Dự đoán giá nhà: Một ví dụ về hồi quy (regression)
- ✓ Tập dữ liệu California Housing Price

```
from keras.datasets import california_housing

(train_data, train_targets), (test_data, test_targets) = (
    california_housing.load_data(version="small")
)
```

```
train_data.shape
```

```
test_data.shape
```

```
train_targets
```

- ✓ Chuẩn bị dữ liệu

```
mean = train_data.mean(axis=0)
std = train_data.std(axis=0)
x_train = (train_data - mean) / std
x_test = (test_data - mean) / std
```

```
y_train = train_targets / 100000
y_test = test_targets / 100000
```

- ✓ Xây dựng mô hình

```
def get_model():
    model = keras.Sequential(
        [
            layers.Dense(64, activation="relu"),
            layers.Dense(64, activation="relu"),
            layers.Dense(1),
        ]
    )
    model.compile(
        optimizer="adam",
        loss="mean_squared_error",
        metrics=["mean_absolute_error"],
    )
    return model
```

- ✓ Kiểm chứng phương pháp sử dụng K-fold validation

```
k = 4
num_val_samples = len(x_train) // k
num_epochs = 50
all_scores = []
for i in range(k):
    print(f"Processing fold #{i + 1}")
```

```

fold_x_val = x_train[i * num_val_samples : (i + 1) * num_val_samples]
fold_y_val = y_train[i * num_val_samples : (i + 1) * num_val_samples]
fold_x_train = np.concatenate(
    [x_train[: i * num_val_samples], x_train[(i + 1) * num_val_samples :]],
    axis=0,
)
fold_y_train = np.concatenate(
    [y_train[: i * num_val_samples], y_train[(i + 1) * num_val_samples :]],
    axis=0,
)
model = get_model()
model.fit(
    fold_x_train,
    fold_y_train,
    epochs=num_epochs,
    batch_size=16,
    verbose=0,
)
scores = model.evaluate(fold_x_val, fold_y_val, verbose=0)
val_loss, val_mae = scores
all_scores.append(val_mae)

```

```

[round(value, 3) for value in all_scores]

```

```

round(np.mean(all_scores), 3)

```

```

k = 4
num_val_samples = len(x_train) // k
num_epochs = 200
all_mae_histories = []
for i in range(k):
    print(f"Processing fold #{i + 1}")
    fold_x_val = x_train[i * num_val_samples : (i + 1) * num_val_samples]
    fold_y_val = y_train[i * num_val_samples : (i + 1) * num_val_samples]
    fold_x_train = np.concatenate(
        [x_train[: i * num_val_samples], x_train[(i + 1) * num_val_samples :]],
        axis=0,
    )
    fold_y_train = np.concatenate(
        [y_train[: i * num_val_samples], y_train[(i + 1) * num_val_samples :]],
        axis=0,
    )
    model = get_model()
    history = model.fit(
        fold_x_train,
        fold_y_train,
        validation_data=(fold_x_val, fold_y_val),
        epochs=num_epochs,
        batch_size=16,
        verbose=0,
    )
    mae_history = history.history["val_mean_absolute_error"]
    all_mae_histories.append(mae_history)

```

```

average_mae_history = [
    np.mean([x[i] for x in all_mae_histories]) for i in range(num_epochs)
]

```

```

epochs = range(1, len(average_mae_history) + 1)
plt.plot(epochs, average_mae_history)
plt.xlabel("Epochs")
plt.ylabel("Validation MAE")
plt.show()

```

```

truncated_mae_history = average_mae_history[10:]
epochs = range(10, len(truncated_mae_history) + 10)
plt.plot(epochs, truncated_mae_history)
plt.xlabel("Epochs")
plt.ylabel("Validation MAE")
plt.show()

```

```
model = get_model()
model.fit(x_train, y_train, epochs=130, batch_size=16, verbose=0)
test_mean_squared_error, test_mean_absolute_error = model.evaluate(
    x_test, y_test
)
```

```
round(test_mean_absolute_error, 3)
```

▼ Tạo dự đoán trên dữ liệu mới

```
predictions = model.predict(x_test)
predictions[0]
```

Tổng kết